IES Maestre de Calatrava

2º CGFS en Desarrollo de Aplicaciones Multiplataforma (DAM)

PROYECTO INTERMODULAR

KAIROS: ASISTENTE DE PRODUCTIVIDAD ADAPTATIVO

(ENTREGA 1: ANÁLISIS Y DISEÑO)

Realizado por: Ismael Manzano León

Tutor: Vanesa Rivero Nieto

Fecha: 20 de Noviembre de 2025

ÍNDICE

DESCRIPCIÓN DEL PROYECTO
- 1. INTRODUCCIÓN
  - a. Objetivos del proyecto
  - b. Contexto
  - c. Estado actual
  - d. Metodología de desarrollo y Tecnologías

REQUISITOS FUNCIONALES
- RF-01. Inicio y Navegación
- RF-02. Gestión de Datos (CRUD)
- RF-03. Persistencia y Sincronización
- RF-04. Validaciones y Control de Errores
- RF-05. Características Adicionales y Lógica de Negocio
- RF-06. Interfaz y Usabilidad

DIAGRAMA DE CASOS DE USO
- 1. REPRESENTACIÓN VISUAL
- 2. ESPECIFICACIÓN DE CASOS DE USO
  - CU-01: Gestión de Tareas (Creación)
  - CU-02: Optimización de Agenda (IA)
  - CU-03: Sincronización de Datos (Offline-First)
  - CU-04: Gestión de Tareas (Completar)

BOCETO DE LA APLICACIÓN (WIREFRAME)
- 1. FLUJO DE NAVEGACIÓN (SCREEN FLOW)
- 2. ESPECIFICACIÓN EN PANTALLA CLAVE
  - A. Pantalla de Inicio / Login (Auth Screen)
  - B. Dashboard Principal (Smart Agenda)
  - C. Vista de Creación de Tarea
  - D. Modo Enfoque (Deep Work)

DESCRIPCIÓN DEL PROYECTO

1. INTRODUCCIÓN

a. Objetivos del proyecto

El objetivo principal de este proyecto es el diseño y desarrollo de KAIROS, una aplicación móvil multiplataforma (Android e iOS) orientada a la optimización de la productividad personal mediante el uso de Inteligencia Artificial.

Como objetivos específicos, se establecen los siguientes:

- Implementar una arquitectura "Offline-First" robusta, garantizando la disponibilidad de los datos sin conexión mediante el uso de bases de datos locales orientadas a objetos.
- Desarrollar un algoritmo de planificación heurística (Smart Scheduling) alojado en un servidor remoto, capaz de reorganizar la agenda del usuario basándose en variables de energía y prioridad.
- Integrar un "Modo Enfoque" que combine técnicas de bloqueo de distracciones con gamificación para incentivar sesiones de trabajo profundo (Deep Work).
- Proporcionar un sistema de analítica de hábitos que permita al usuario visualizar patrones de comportamiento y corregir tendencias de procrastinación.

b. Contexto

En la era de la hiperconexión, la gestión del tiempo ya no ocurre exclusivamente frente a un escritorio de oficina. El usuario moderno, estudiante o profesional, se enfrenta a una fragmentación de la atención en movilidad: ideas que surgen en el transporte, tareas que se recuerdan en el gimnasio o notificaciones que interrumpen el tiempo de ocio.

Las herramientas de escritorio obligan al usuario a "sentarse a planificar", creando una barrera de entrada. KAIROS nace para integrarse en el dispositivo que acompaña al usuario las 24 horas: su smartphone. Su objetivo no es ser un contenedor pasivo de datos (hoja de cálculo), sino una herramienta de intervención activa que utiliza sensores nativos, notificaciones push y la inmediatez táctil para capturar tareas y bloquear distracciones (Modo Enfoque) directamente en la fuente de la interrupción.

c. Estado actual

El mercado de la productividad se divide actualmente en dos extremos:

1. Gestores de Listas Simples (Móvil): Apps como Google Tasks o Apple Reminders, que son rápidas pero carecen de inteligencia para priorizar.
2. Planificadores Inteligentes (Web/Escritorio): Soluciones como Motion o Reclaim.ai que reorganizan agendas con IA, pero están diseñadas como dashboards complejos de escritorio, inaccesibles o poco usables cuando el usuario está lejos de su ordenador.

KAIROS ocupa el vacío existente entre ambos: una aplicación nativa móvil que trae la potencia algorítmica de las plataformas de escritorio a una interfaz táctil simplificada. La elección de una arquitectura Mobile Offline-First no es estética, sino funcional: garantiza que la asistencia esté disponible en "tiempo de vida" (offline en aviones, metro, zonas rurales), algo que una solución web (SaaS) dependiente de conexión constante no puede garantizar.

d. Metodología de desarrollo y Tecnologías

Para la ejecución del proyecto se adoptará una metodología ágil SCRUM, estructurando el desarrollo en sprints de dos semanas para permitir la iteración rápida y la adaptación continua del algoritmo de recomendación. El control de versiones y la gestión del backlog se realizarán mediante GitHub Projects, asegurando una trazabilidad completa de las tareas.

La arquitectura tecnológica seleccionada (Clean Architecture) garantiza escalabilidad, consistencia y rendimiento profesional:

- Frontend (Aplicación Móvil): Se utilizará Flutter (Dart). Esta elección permite desplegar código nativo compilado tanto en Android como en iOS desde una única base de código, asegurando una interfaz fluida (60fps) y una experiencia de usuario reactiva.
- Backend (API y Lógica): Se implementará una API RESTful con Python (FastAPI). Python es el estándar en la industria para el procesamiento de datos y permitirá la integración nativa de librerías de IA (como Scikit-learn o Pandas) para el motor de recomendación.
- Persistencia de Datos (Offline-First):
  - Local (Dispositivo): Se utilizará Realm Database. Al ser una base de datos orientada a objetos y de alto rendimiento (zero-copy), elimina la necesidad de mapeo O/R (ORM) en el dispositivo, permitiendo una persistencia extremadamente rápida y compleja ideal para el funcionamiento sin conexión.
  - Nube (Central): PostgreSQL servirá como fuente de verdad en el servidor.
  - Estrategia de Sincronización: Se implementará el Patrón Repository en la capa de dominio de la aplicación móvil. Este repositorio actuará como árbitro, encargándose de la serialización de los objetos locales de Realm a estructuras JSON compatibles con la API REST de PostgreSQL, garantizando la integridad de datos entre el modelo orientado a objetos (móvil) y el modelo relacional (nube).

REQUISITOS FUNCIONALES

El sistema KAIROS cumplirá con las siguientes especificaciones funcionales, clasificadas según su ámbito de aplicación en la arquitectura del software.

RF-01. Inicio y Navegación

- **RF-01.1 Pantalla de Carga (Splash Screen)**: Al iniciar la aplicación, el sistema mostrará una pantalla de bienvenida con el logotipo de KAIROS mientras se inicializan los servicios de base de datos local (Realm) y se verifica el estado de la sesión.
- **RF-01.2 Autenticación**: El sistema requerirá que el usuario se identifique mediante credenciales (email/contraseña) para acceder a las funciones de sincronización en la nube. Se gestionará mediante tokens de sesión (JWT) contra la API REST.
- **RF-01.3 Navegación Principal**: La interfaz implementará una barra de navegación inferior persistente que permitirá transitar instantáneamente entre los módulos principales: "Agenda Hoy", "Lista de Tareas", "Modo Enfoque" y "Perfil/Estadísticas".

RF-02. Gestión de Datos (CRUD)

- **RF-02.1 Creación de Tareas**: El usuario podrá registrar nuevas tareas especificando atributos obligatorios (título) y opcionales (descripción, fecha límite).
- **RF-02.2 Atributos Paramétricos**: Para alimentar el algoritmo de IA, el sistema permitirá asignar metadatos específicos a cada tarea:
  - Prioridad: Alta, Media, Baja.
  - Nivel de Energía: Estimación del esfuerzo cognitivo requerido (1 a 5).
- **RF-02.3 Edición y Eliminación**: El usuario podrá modificar cualquier atributo de una tarea existente o eliminarla definitivamente de la base de datos.
- **RF-02.4 Marcado de Completitud**: El sistema permitirá marcar tareas como "Completadas", lo cual actualizará las estadísticas de productividad del usuario.

RF-03. Persistencia y Sincronización

- **RF-03.1 Persistencia Local (Offline-First)**: Todos los datos generados (tareas, sesiones de enfoque) se guardarán inmediatamente en la base de datos local Realm dentro del dispositivo, garantizando que la información persista tras cerrar la aplicación o reiniciar el terminal.
- **RF-03.2 Sincronización Remota**: Al detectar conexión a internet, el sistema sincronizará los cambios locales con la base de datos central (PostgreSQL) a través de la API, resolviendo conflictos de versiones si fuera necesario.

RF-04. Validaciones y Control de Errores

- **RF-04.1 Integridad de Formularios**: El sistema impedirá guardar una tarea si el campo "Título" está vacío, mostrando una alerta visual inmediata.
- **RF-04.2 Validación de Tipos**: Se validará que los campos numéricos (duración, energía) y de fecha respeten el formato correcto antes de procesar la solicitud.
- **RF-04.3 Feedback de Error**: En caso de fallo en la comunicación con la API (Error 500/404) o falta de conexión, el sistema notificará al usuario mediante mensajes no intrusivos (Snackbars), asegurando que la experiencia de uso no se interrumpa.

RF-05. Características Adicionales y Lógica de Negocio

- **RF-05.1 Algoritmo de Planificación (Smart Scheduling)**: El sistema dispondrá de una función "Optimizar Día" que enviará las tareas pendientes al servidor. El backend reordenará la lista basándose en la prioridad, fecha límite y energía disponible, devolviendo una agenda sugerida al usuario.

- **RF-05.2 Modo Enfoque (Temporizador)**: El sistema incluirá un cronómetro de concentración (técnica Pomodoro) que registrará el tiempo dedicado a una tarea específica.
- **RF-05.3 Visualización de Métricas**: El sistema generará gráficos de rendimiento basados en el historial de tareas completadas y sesiones de enfoque almacenadas.

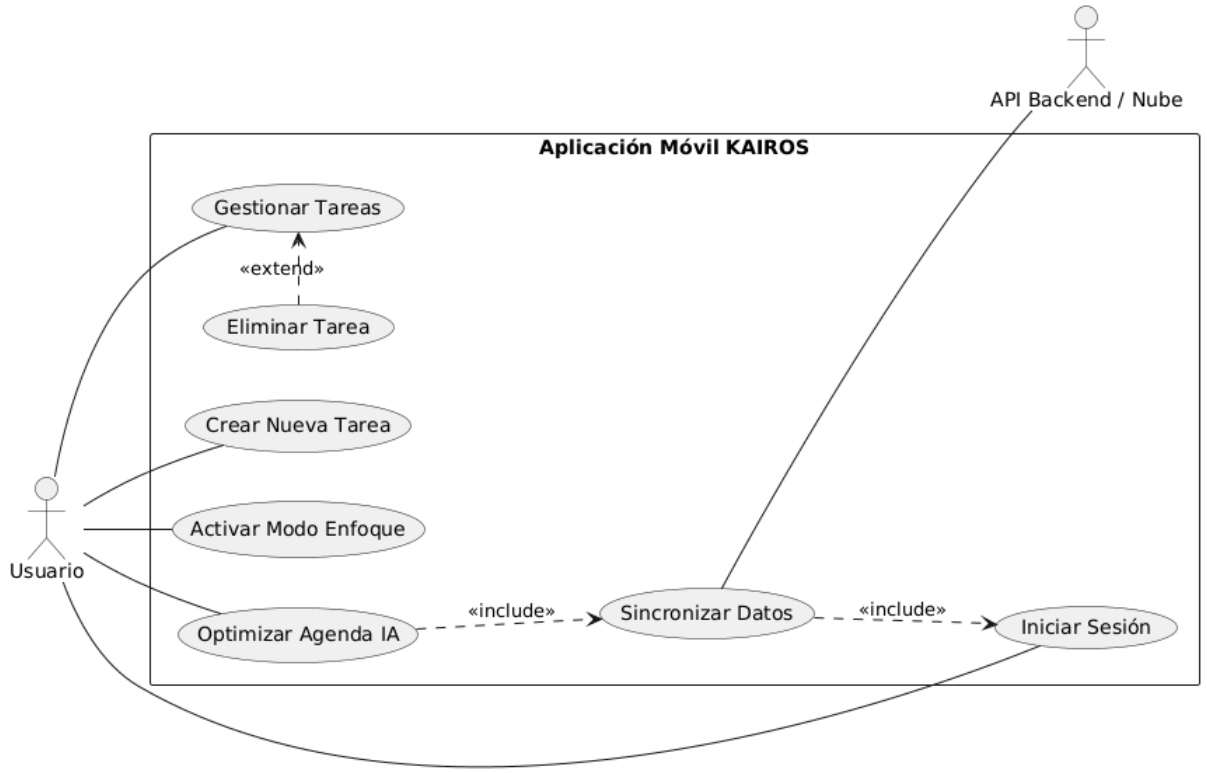
RF-06. Interfaz y Usabilidad

- **RF-06.1 Diseño Adaptativo**: La interfaz se construirá mediante Widgets de Flutter que se adaptarán visualmente a las guías de estilo nativas de Android (Material Design 3) e iOS (Cupertino).
- **RF-06.2 Interactividad**: Los elementos de la lista permitirán interacciones gestuales (gestos de deslizamiento o swipe) para completar o borrar tareas rápidamente.

DIAGRAMA DE CASOS DE USO

A continuación se modela la interacción del usuario con el sistema KAIROS, identificando los procesos clave y sus dependencias.

1. REPRESENTACIÓN VISUAL



2. ESPECIFICACIÓN DE CASOS DE USO

De acuerdo con la metodología de documentación, se detallan a continuación los flujos de los casos de uso principales que involucran la lógica de negocio crítica (Offline-First e Inteligencia Artificial).

CU-01: Gestión de Tareas (Creación)

Este caso de uso describe cómo el sistema maneja la creación de datos asegurando la persistencia local inmediata (Realm) antes de cualquier sincronización.

Campo	**Descripción**
Identificador	CU-01
Nombre	Crear Nueva Tarea
Actores	Usuario (Principal)
Precondiciones	La aplicación debe estar iniciada y en la pantalla "Lista de Tareas".
Flujo Principal	1. El Usuario selecciona la opción "Añadir Tarea" (+).2. El Sistema despliega el formulario de creación.3. El Usuario introduce Título (obligatorio), Prioridad y Energía.4. El Sistema valida que el campo Título no esté vacío.5. El Sistema guarda el objeto tarea en la base de datos local (Realm).6. El Sistema muestra la tarea en la lista actualizada.
Flujo Alternativo	**4a. Título Vacío:**1. Si el usuario intenta guardar sin título, el sistema bloquea la acción.2. Se muestra un mensaje de error "El título es obligatorio".
Postcondiciones	La tarea queda registrada en el dispositivo localmente, pendiente de sincronización (flag isSynced = false) si no hay conexión.

CU-02: Optimización de Agenda (IA)

Este caso de uso modela la interacción con el algoritmo heurístico remoto para la reorganización inteligente de tareas.

Campo	**Descripción**
Identificador	CU-02
Nombre	Optimizar Agenda con IA
Actores	Usuario (Principal), API Backend (Secundario)
Precondiciones	El usuario debe tener al menos 3 tareas pendientes y conexión a internet activa.
Flujo Principal	1. El Usuario pulsa el botón "Optimizar mi Día".2. El Sistema verifica la conectividad a internet.3. El Sistema recopila las tareas pendientes locales y las serializa a JSON.4. El Sistema envía la carga útil (payload) al endpoint /optimize de la API.5. La API procesa los datos mediante el algoritmo heurístico y devuelve el orden sugerido.6. El Sistema recibe la respuesta y actualiza el orden visual de la lista en la UI.
Flujo Alternativo	**2a. Sin Conexión:**1. El Sistema detecta falta de red.2. Muestra notificación: "Función no disponible offline".**5a. Error en Servidor:**1. La API devuelve error 500 o Timeout.2. El Sistema mantiene el orden original y notifica "Inténtelo más tarde".
Postcondiciones	Las tareas se reordenan visualmente según la prioridad calculada por el servidor.

CU-03: Sincronización de Datos (Offline-First)

Este caso de uso detalla el proceso de mantener la coherencia de datos entre el dispositivo local y el servidor, una vez que la conexión a internet está disponible.

Campo	**Descripción**
Identificador	CU-03
Nombre	Sincronizar Datos
Actores	Usuario (Pasivo), Sistema (Principal)
Precondiciones	El usuario ha modificado o creado datos (ej. tareas) sin conexión, o se ha detectado una nueva conexión a internet.
Flujo Principal	1. El Sistema detecta conexión a internet (al inicio o durante el uso).2. El Sistema identifica los objetos locales con la bandera isSynced = false.3. El Sistema envía los datos pendientes al servidor a través de la API REST.4. El Servidor recibe los datos, los valida y los registra en PostgreSQL.5. El Servidor devuelve confirmación de éxito.6. El Sistema actualiza el objeto local en Realm estableciendo isSynced = true.7. El Sistema descarga cualquier actualización pendiente del servidor y la aplica localmente.
Flujo Alternativo	**3a. Conflicto de Versiones:**1. El servidor detecta que el registro local tiene una versión más antigua que la almacenada.2. Se utiliza una política de "Last Write Wins" (la última modificación gana) o se fusionan los datos si es

Campo	Descripción
	posible.3. Se notifica al usuario si la fusión no es trivial.
Postcondiciones	Los datos locales y remotos son coherentes. Todos los objetos locales tienen isSynced = true.

CU-04: Gestión de Tareas (Completar)

Este caso de uso modela el proceso de finalización de una tarea, lo cual es fundamental para el cálculo de estadísticas de productividad.

Campo	Descripción
Identificador	CU-04
Nombre	Marcar Tarea como Completada
Actores	Usuario (Principal)
Precondiciones	La aplicación está en la pantalla "Lista de Tareas" y existe al menos una tarea pendiente.
Flujo Principal	1. El Usuario realiza un gesto (ej. *swipe* o pulsa un checkbox) sobre una tarea.2. El Sistema marca la tarea como completada en la base de datos local (Realm), registrando la fecha/hora de finalización.3. El Sistema elimina visualmente la tarea de la lista de pendientes (o la mueve a una lista de completadas).4. El Sistema marca el objeto como pendiente de sincronización (isSynced = false).5. (El sistema continúa con el CU-03 de sincronización si hay conexión).

Campo	**Descripción**
Flujo Alternativo	No aplica. La acción debe ser siempre posible localmente.
Postcondiciones	La tarea se registra como finalizada y los datos de productividad quedan actualizados en el dispositivo.

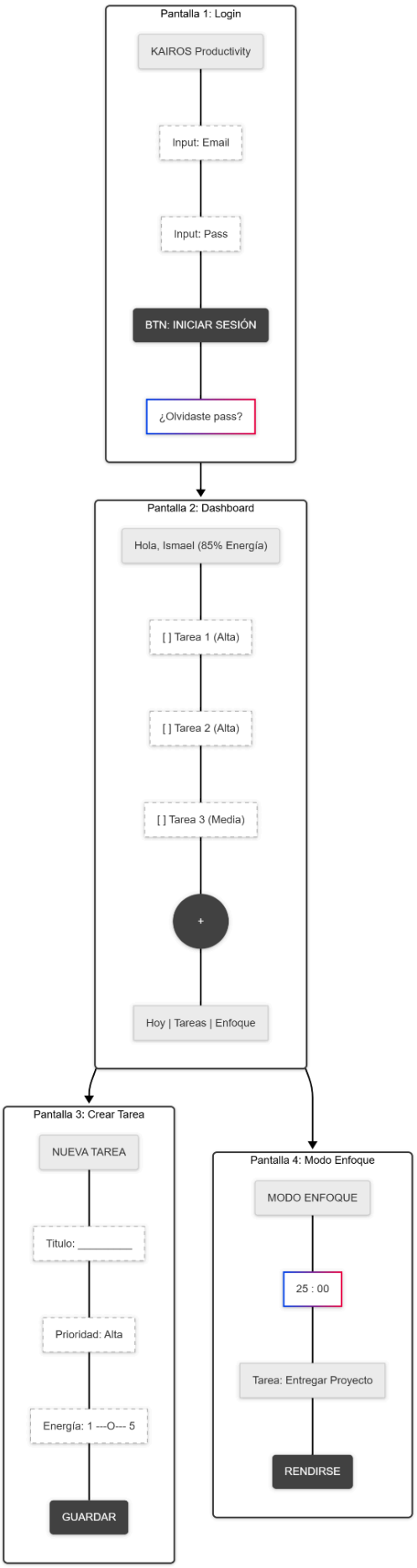
BOCETO DE LA APLICACIÓN (WIREFRAME)

El diseño de la interfaz de usuario (UI) de KAIROS se ha concebido siguiendo los principios de Material Design 3, priorizando la claridad, la accesibilidad y la minimización de la carga cognitiva para fomentar la productividad.

A continuación se detalla el flujo de navegación y la disposición de los elementos en las pantallas principales, cumpliendo con los requisitos funcionales descritos en la Sección 2.

1. FLUJO DE NAVEGACIÓN (SCREEN FLOW)

El siguiente esquema representa la arquitectura de la información y las transiciones principales entre vistas. Se destacan las rutas de "Login", "Gestión de Tareas" y el acceso al "Algoritmo de Optimización".


Pantalla 1: Login
KAIROS Productivity
Input: Email
Input: Pass
BTN: INICIAR SESIÓN
¿Olvidaste pass?
Pantalla 2: Dashboard
Hola, Ismael (85% Energía)
[ ] Tarea 1 (Alta)
[ ] Tarea 2 (Alta)
[ ] Tarea 3 (Media)
+
Hoy | Tareas | Enfoque
Pantalla 3: Crear Tarea
NUEVA TAREA
Título: _____
Prioridad: Alta
Energía: 1 ---O--- 5
GUARDAR
Pantalla 4: Modo Enfoque
MODO ENFOQUE
25 : 00
Tarea: Entregar Proyecto
RENDIRSE

Nota técnica sobre el flujo: Las flechas indican la navegación explícita. El acceso al "Modo Enfoque" es una navegación modal (pantalla completa) para bloquear distracciones, mientras que la "Optimización" es un estado transitorio que retorna al Dashboard actualizado.

2. ESPECIFICACIÓN EN PANTALLA CLAVE

Se detallan los componentes de las vistas críticas para el MVP:

A. Pantalla de Inicio / Login (Auth Screen)

- Objetivo: Autenticación segura para permitir la sincronización con la nube (PostgreSQL).
- Elementos:
  - Campo de texto: Correo electrónico (con validación de formato).
  - Campo de texto: Contraseña (oculta por defecto).
  - Botón Principal (CTA): "Iniciar Sesión / Sincronizar".
  - Feedback: Indicador de estado de conexión (Online/Offline).

B. Dashboard Principal (Smart Agenda)

- Objetivo: Vista central donde el usuario gestiona su día.
- Elementos:
  - Header: Saludo personalizado y métrica de "Energía Actual".
  - Lista de Tareas: Elementos ordenados por prioridad. Cada ítem muestra Título, Etiqueta de Prioridad (Alta/Media/Baja) y Checkbox de completado.
  - Botón de Acción Flotante (FAB): Icono "+" para crear nuevas tareas rápidamente (CRUD).
  - Botón de Acción Secundaria: "Optimizar Día" (Icono de varita mágica/IA) situado en la barra superior. Al pulsarlo, dispara el algoritmo heurístico (RF-05.1).

C. Vista de Creación de Tarea

- Objetivo: Formulario de entrada de datos.
- Elementos:

- Campos: Título (Obligatorio), Descripción, Fecha límite.
- Selectores Paramétricos: Slider de Energía (1-5) y Selector de Prioridad.
- Botones: "Cancelar" y "Guardar" (Persistencia en Realm).

D. Modo Enfoque (Deep Work)

- Objetivo: Maximizar la concentración mediante aislamiento visual.
- Elementos:
  - Temporizador Central (Cuenta regresiva grande).
  - Control único: Botón "Rendirse" o "Pausar".
  - Fondo: Minimalista/Oscuro para reducir fatiga visual.